

# Características de repositórios populares

**Pedro Negri Leão Lambert**

**11/03/2026**

**Link para repo/dados:**

<https://github.com/Pedro-nll/LabExperimentacaoDeSoftware/tree/enunciado1>

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>Características de repositórios populares.....</b> | <b>1</b> |
| Introdução.....                                       | 2        |
| Contextualização.....                                 | 2        |
| Restrições e riscos.....                              | 3        |
| Problema foco do experimento.....                     | 3        |
| Questões de Pesquisa.....                             | 3        |
| Hipóteses.....                                        | 3        |
| Objetivo.....                                         | 4        |
| Objetivo principal.....                               | 4        |
| Objetivos específicos.....                            | 4        |
| Metodologia.....                                      | 4        |
| Passo a passo do experimento.....                     | 5        |
| Decisões.....                                         | 5        |
| Materiais utilizados.....                             | 5        |
| Métodos utilizados.....                               | 5        |
| Métricas e suas unidades.....                         | 6        |
| Visualização dos Resultados.....                      | 6        |
| Discussão dos Resultados.....                         | 11       |
| Conclusão.....                                        | 12       |
| Reprodutibilidade.....                                | 12       |

# Introdução

## Contextualização

Projetos open source desempenham um papel fundamental no desenvolvimento de software moderno. Grande parte da infraestrutura digital atual depende de bibliotecas, frameworks e ferramentas mantidas por comunidades distribuídas de desenvolvedores. Nesse contexto, compreender as características dos projetos open source mais populares pode fornecer insights importantes sobre práticas de desenvolvimento, manutenção e colaboração em larga escala.

Plataformas como o GitHub permitem a mineração de grandes volumes de dados sobre repositórios de software. Métricas como idade do projeto, volume de contribuições externas, frequência de releases, frequência de atualização e gerenciamento de issues podem revelar padrões relevantes sobre a maturidade e a atividade desses projetos.

Este experimento utiliza dados públicos do GitHub para analisar características de repositórios populares e identificar padrões relacionados ao desenvolvimento de software open source em larga escala.

## Restrições e riscos

Algumas limitações devem ser consideradas neste experimento. Primeiramente, a popularidade dos projetos foi estimada com base no número de estrelas no GitHub, o que não necessariamente reflete o uso real ou a importância prática do projeto.

Além disso, alguns repositórios populares não representam sistemas de software propriamente ditos, podendo ser coleções de recursos, listas de materiais ou conteúdo educacional. Para reduzir esse viés, foram removidos manualmente 103 repositórios considerados fora do escopo de engenharia de software.

Por fim, a análise baseia-se apenas em metadados disponíveis na API do GitHub, não incluindo análise qualitativa do código ou da comunidade de contribuidores.

## Problema foco do experimento

Este experimento busca compreender quais são as principais características estruturais e de atividade dos repositórios open source mais populares do GitHub.

## Questões de Pesquisa

RQ01 – Sistemas populares são maduros/antigos?

RQ02 – Sistemas populares recebem muita contribuição externa?

RQ03 – Sistemas populares lançam releases com frequência?

RQ04 – Sistemas populares são atualizados com frequência?

RQ05 – Sistemas populares são escritos nas linguagens mais populares?

RQ06 – Sistemas populares possuem um alto percentual de issues fechadas?

RQ07 – Sistemas escritos em linguagens mais populares recebem mais contribuições externas, lançam mais releases e são atualizados com mais frequência?

## Hipóteses

H1 – Repositórios populares tendem a ser relativamente antigos e maduros, acumulando vários anos de desenvolvimento.

H2 – Repositórios populares recebem grande volume de contribuições externas ao longo do tempo.

H3 – Projetos populares costumam publicar releases com frequência.

H4 – Repositórios populares são frequentemente atualizados e mantidos ativamente.

H5 – A maioria dos repositórios populares é desenvolvida em linguagens amplamente utilizadas, como Python, JavaScript e TypeScript.

H6 – Repositórios populares apresentam alta taxa de fechamento de issues.

H7 – A atividade de desenvolvimento pode variar significativamente entre projetos e não necessariamente depende da popularidade da linguagem utilizada.

# Objetivo

## Objetivo principal

Analisar características estruturais e de atividade dos repositórios open source mais populares do GitHub.

## Objetivos específicos

- Identificar a idade típica dos repositórios populares
- Avaliar o volume de contribuições externas
- Analisar a frequência de releases
- Verificar a frequência de atualização dos projetos
- Identificar as linguagens predominantes
- Avaliar o gerenciamento de issues
- Investigar possíveis relações entre linguagem e atividade de desenvolvimento

# Metodologia

## Passo a passo do experimento

1. Coleta dos 1000 repositórios com maior número de estrelas no GitHub utilizando consultas à API GraphQL.
2. Extração das métricas necessárias para responder às questões de pesquisa.
3. Remoção de 103 repositórios que não representam projetos de software.
4. Armazenamento dos dados em arquivo CSV.
5. Análise estatística dos dados coletados.
6. Geração de visualizações e gráficos para interpretação dos resultados.

Após a filtragem, o dataset final utilizado na análise contém 897 repositórios.

## Decisões

Durante o experimento foram tomadas algumas decisões metodológicas importantes:

- utilizar a mediana como principal medida de tendência central, devido à presença de valores extremos;
- remover repositórios que não representam sistemas de software;

## Materiais utilizados

- Linguagem de programação Python
- GitHub GraphQL API
- Bibliotecas de análise de dados em Python
- Ambiente de execução local

## Métodos utilizados

Foram aplicadas técnicas de mineração de repositórios de software (MSR) para coletar e analisar dados públicos do GitHub.

A análise estatística incluiu:

- média
- mediana
- desvio padrão
- análise de distribuição

Os resultados foram representados graficamente para facilitar a interpretação.

## Métricas e suas unidades

Idade do repositório → anos

Pull requests aceitas → número de PRs

Total de releases → número de versões

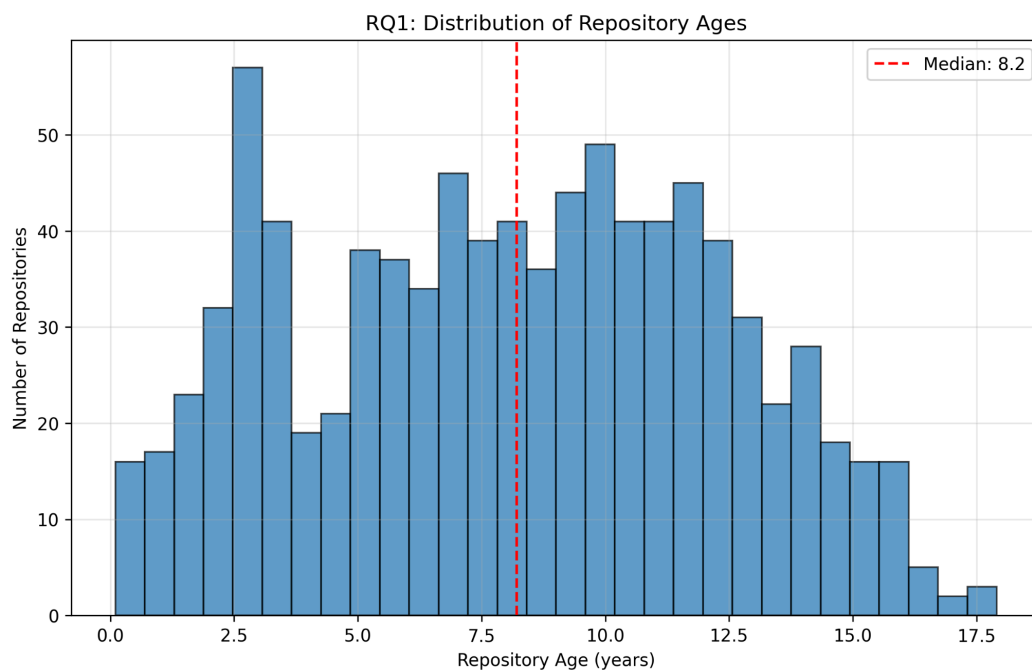
Dias desde última atualização → dias

Linguagem primária → categoria

Razão de issues fechadas → proporção

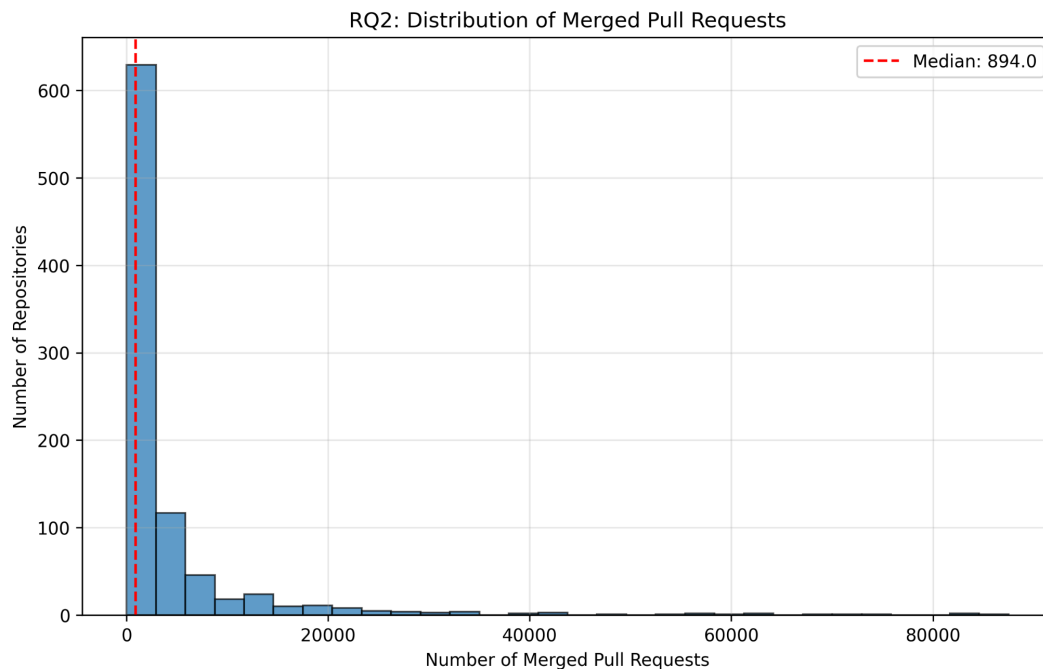
## Visualização dos Resultados

## Idade dos repositórios



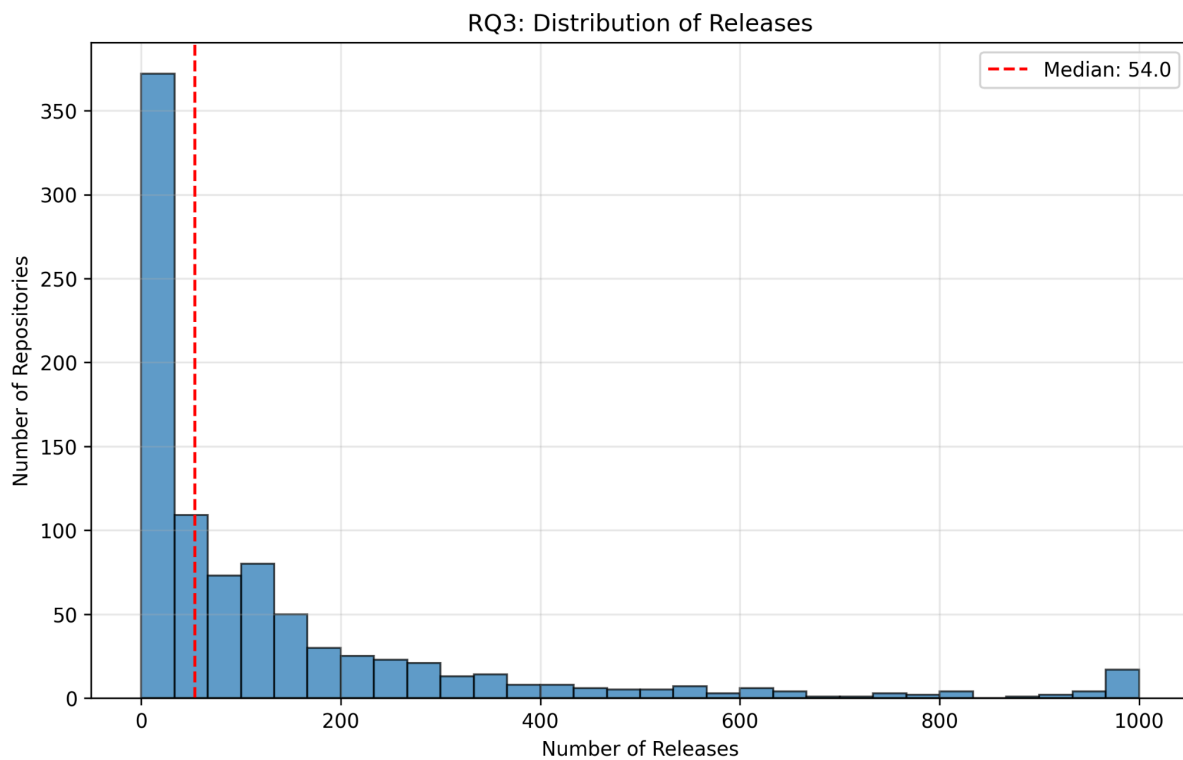
A distribuição de idade dos repositórios mostra uma mediana de aproximadamente 8.2 anos, indicando que a maioria dos projetos populares possui um longo histórico de desenvolvimento.

## Contribuições externa



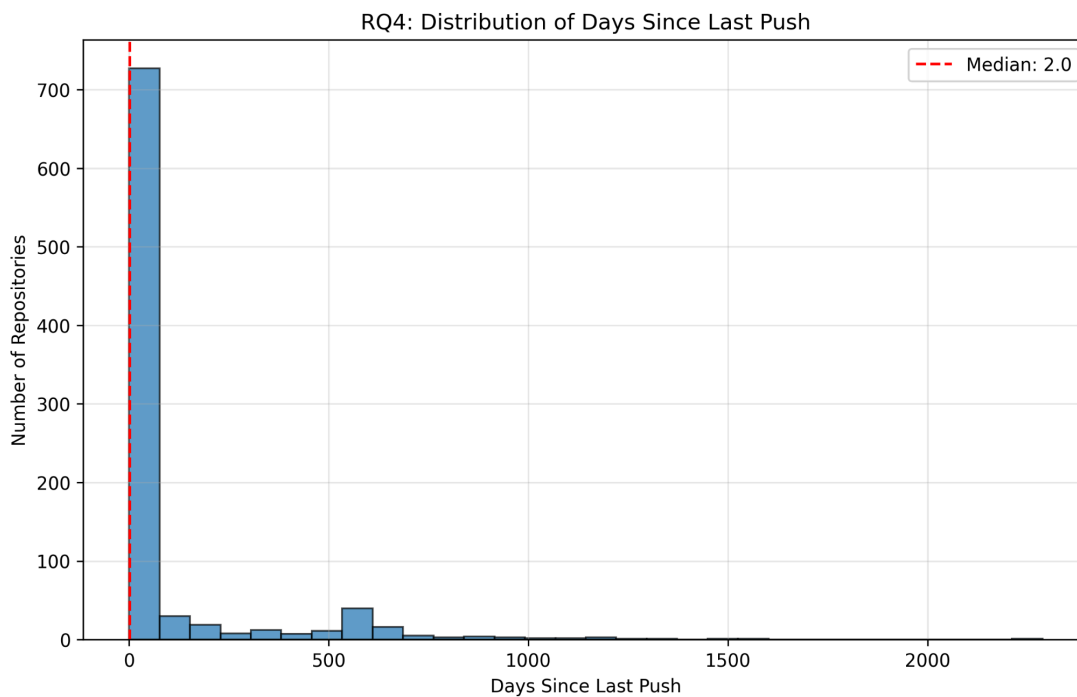
A mediana de 894 pull requests aceitas indica um volume significativo de contribuições externas nos projetos analisados.

## Releases



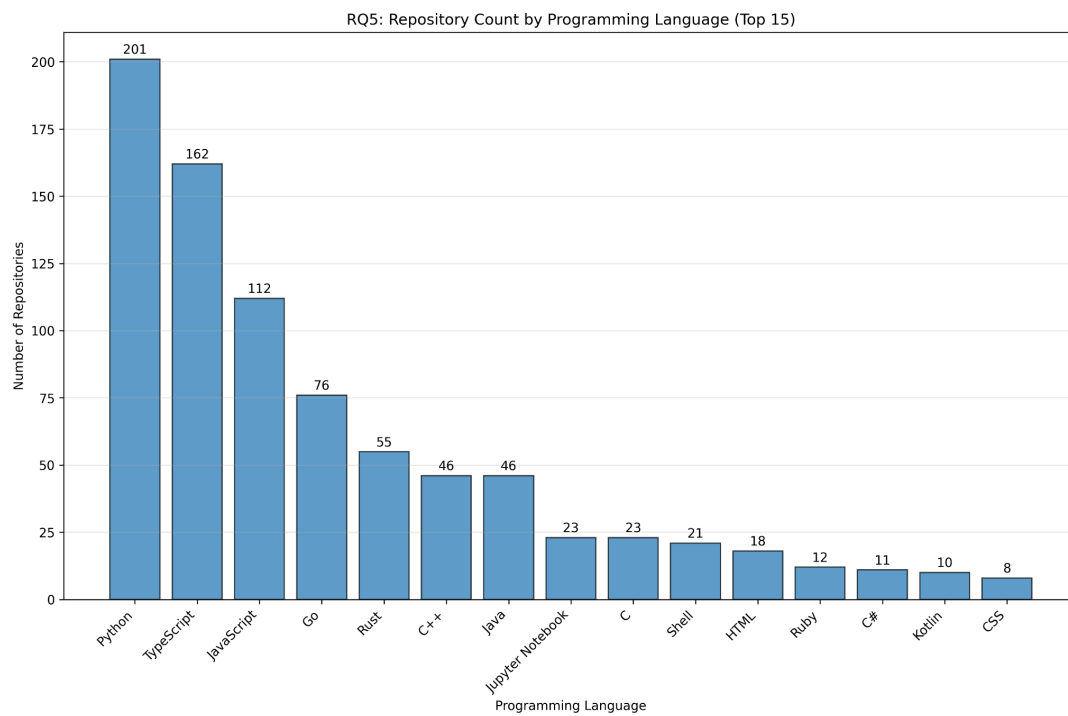
A mediana observada foi de 54 releases, sugerindo que projetos populares costumam publicar novas versões regularmente.

## Atualizações recentes



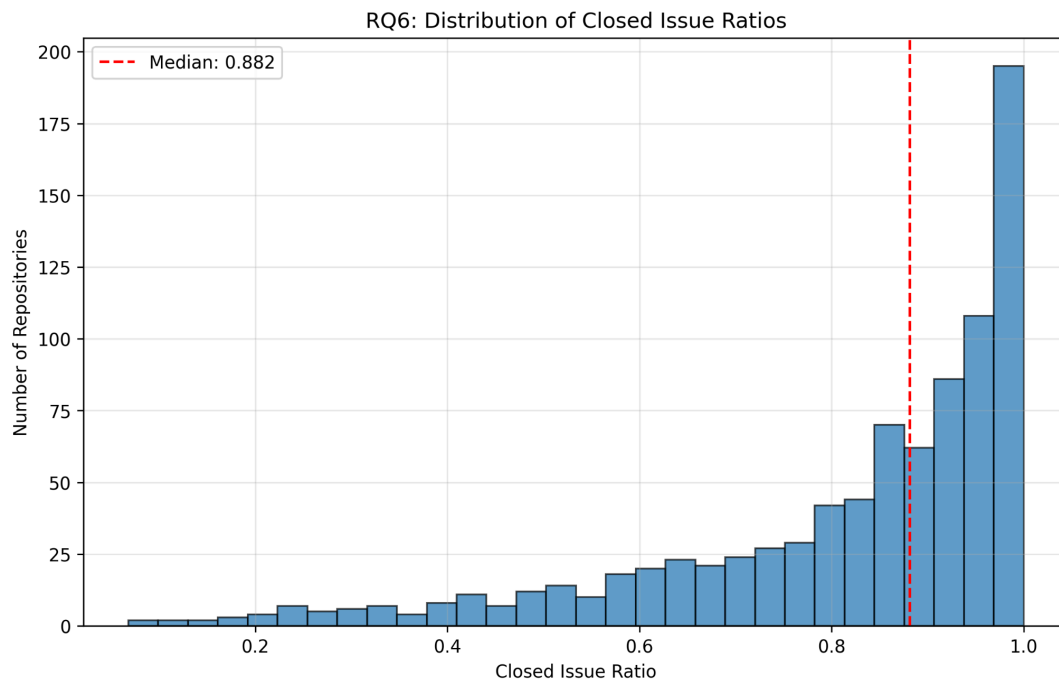
A mediana de 2 dias desde a última atualização indica que a maioria dos projetos analisados permanece ativa.

## Linguagens utilizadas



Observa-se predominância de linguagens populares como Python, TypeScript, JavaScript, Go e Rust.

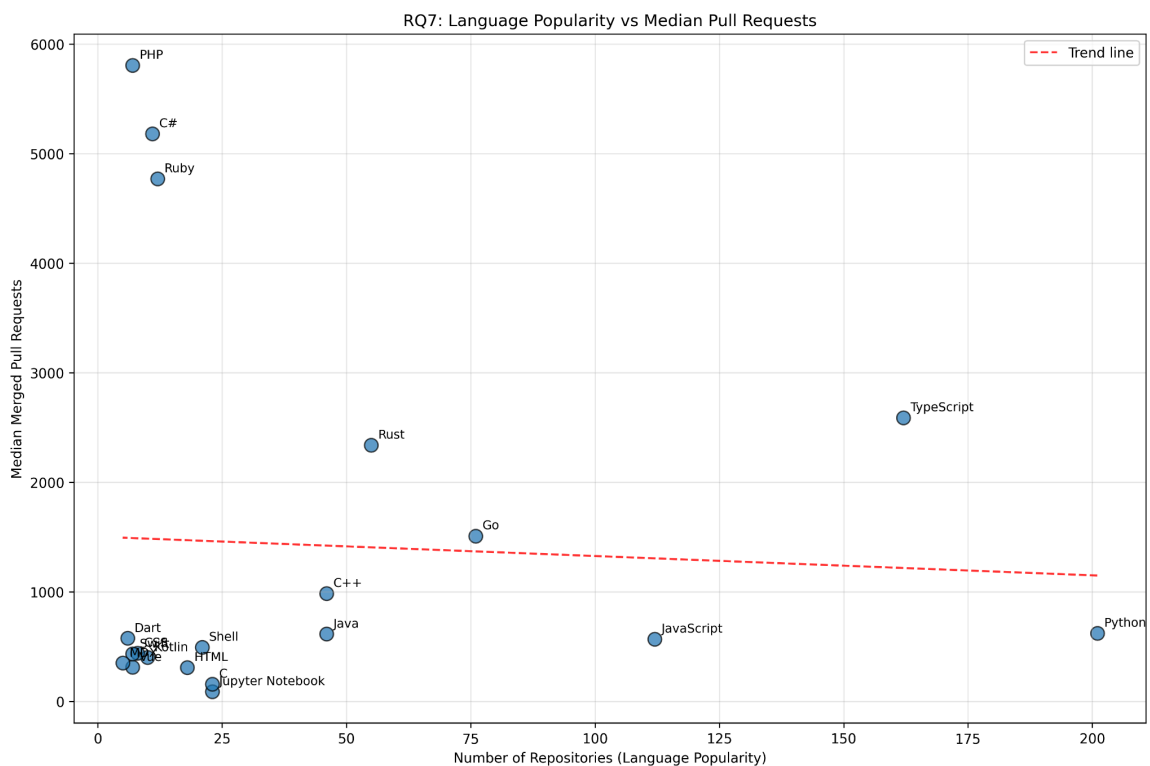
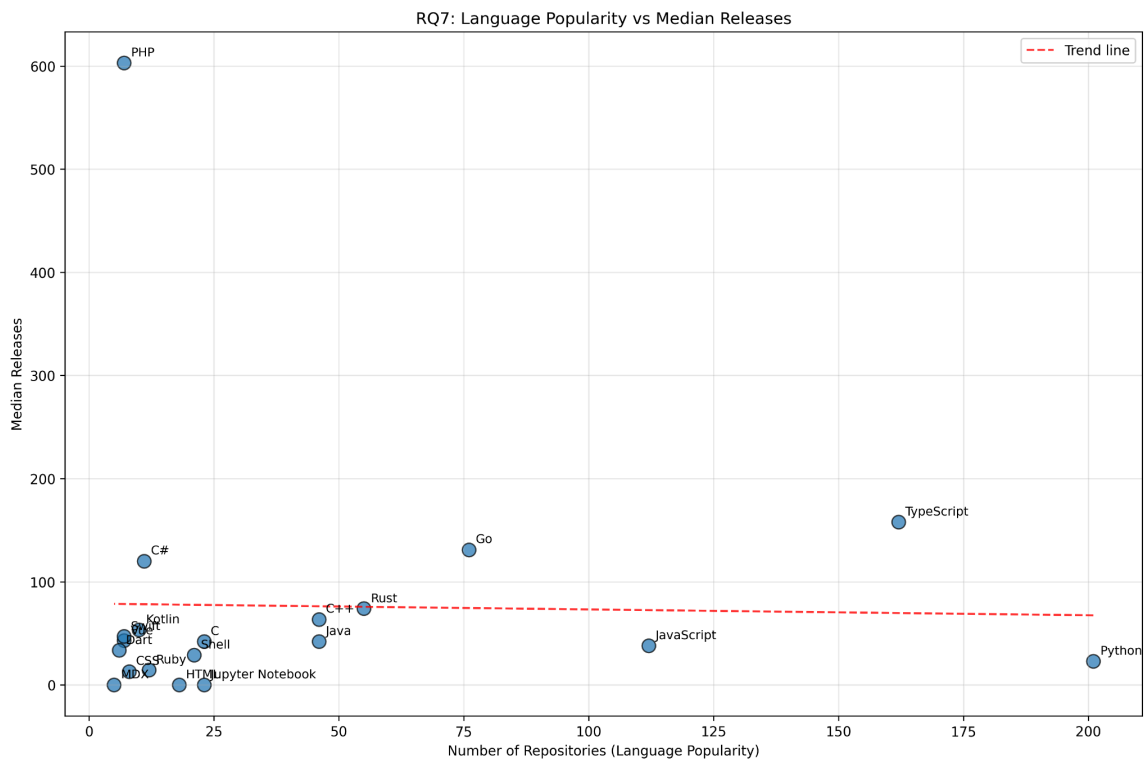
## Razão de issues fechadas



A mediana da razão de issues fechadas é de aproximadamente 88%, indicando que grande parte das issues abertas é posteriormente resolvida.



## Relação entre linguagem e atividade



Os gráficos mostram grande variação na atividade de desenvolvimento entre diferentes linguagens.

## Discussão dos Resultados

A análise da idade dos repositórios confirma a hipótese H1. A mediana de aproximadamente 8 anos indica que projetos populares tendem a possuir longo histórico de desenvolvimento, sugerindo maturidade e estabilidade.

A hipótese H2 também é corroborada pelos resultados. A mediana de 894 pull requests aceitas demonstra alto nível de participação da comunidade nesses projetos.

Entretanto, observa-se uma diferença significativa entre média e mediana no número de pull requests (média  $\approx 4248$ , mediana  $\approx 894$ ). Esse comportamento indica uma distribuição altamente assimétrica, na qual poucos projetos concentram uma quantidade extremamente alta de contribuições, enquanto a maioria apresenta valores menores. Esse padrão é comum em ecossistemas open source e sugere a presença de uma distribuição de cauda longa.

A hipótese H3 também é confirmada. A mediana de 54 releases indica que projetos populares frequentemente utilizam processos estruturados de versionamento e distribuição de software.

Em relação à frequência de atualização (H4), os resultados indicam que a maioria dos projetos permanece ativa, com atualizações recentes mesmo após muitos anos de desenvolvimento. Isso sugere manutenção contínua e forte engajamento das comunidades de desenvolvimento.

A hipótese H5 também é suportada pelos dados. As linguagens mais frequentes nos repositórios analisados incluem Python, TypeScript e JavaScript, que também figuram entre as linguagens mais populares na indústria.

A análise da razão de issues fechadas confirma H6. A mediana de 88% indica que a maioria das issues abertas é posteriormente resolvida, sugerindo processos ativos de manutenção e gestão da comunidade.

Por fim, a análise da RQ07 indica que não existe uma relação clara entre popularidade da linguagem e intensidade de atividade do projeto. Alguns projetos em linguagens menos frequentes apresentam valores muito altos de pull requests e releases. Isso sugere que fatores como tamanho da comunidade, objetivo do projeto e adoção na indústria podem ter maior influência na atividade de desenvolvimento do que a linguagem utilizada.

Outro resultado relevante é a grande diversidade de linguagens presentes entre os projetos populares. Mesmo entre os repositórios mais populares do GitHub observa-se a presença de dezenas de linguagens diferentes, indicando diversidade tecnológica significativa no ecossistema open source.

## Conclusão

Este experimento analisou características estruturais e de atividade de repositórios open source populares no GitHub.

Os resultados indicam que projetos populares geralmente apresentam:

- longa duração de desenvolvimento
- forte participação da comunidade
- ciclos frequentes de releases
- manutenção ativa
- alto percentual de resolução de issues

Além disso, observou-se que a atividade de desenvolvimento não depende exclusivamente da linguagem utilizada, mas parece estar mais relacionada às características individuais de cada projeto e à sua comunidade.

Esses resultados estão alinhados com estudos sobre comunidades open source. Pesquisas como a de Crowston et al. (2012) mostram que projetos open source bem-sucedidos tendem a possuir comunidades ativas e ciclos contínuos de evolução do software.

Como trabalho futuro, seria interessante expandir este experimento incluindo:

- análise da rede de contribuidores
- análise temporal das contribuições
- estudo da relação entre tamanho da comunidade e sucesso do projeto
- comparação com projetos menos populares.

## Reprodutibilidade

O experimento pode ser reproduzido utilizando os scripts disponíveis no repositório do projeto.

O processo ocorre em duas etapas principais.

### **Mineração dos dados**

O script:

**[enunciado1/main.py](#)**

realiza consultas à API GraphQL do GitHub para coletar os metadados dos repositórios e exporta os resultados para um arquivo CSV.

### **Análise dos dados**

O script:

**[enunciado1/analysis.py](#)**

lê o arquivo CSV gerado, calcula as métricas estatísticas e produz os gráficos utilizados neste relatório.

Executando esses dois scripts na mesma ordem é possível reproduzir integralmente os resultados apresentados neste trabalho.